IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

Name: Group 80	
Student Reference Number: Group 80	

Module Code: PUSL 2021	Module Name: Computing Group Project
Coursework Title: Functional & Technical Specification Report Submission	
Deadline Date: 18/01/2026	Member of staff responsible for coursework: Mr. Anton Jayakodi
Programme: BSc (Hons) Data Science , BSc (Hons) Artificial Intelligence	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

Name	Student ID
SAP Prabhashana	10967100
Rubasinghe D D	10967104
YUS De Yapa	10967122
NPMMD Pathirana	10967063
RRI Dissanayaka	10967081
AVJA Amarasekera	10967076
PGT Sewmini	10967111

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group:

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***
Signed :

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.

I *have used/not used translation software.

If used, please state name of software……

Overall mark _____% **Assessors Initials _____** **Date – 31/10/2025**

Table of Contents

CHAPTER 1: INTRODUCTION 5

1.1 PURPOSE OF THE DOCUMENT 5
1.2 SCOPE OF THE SYSTEM 5
1.2.1 IN-SCOPE 5
1.2.2 OUT-OF-SCOPE (WHAT LINKABLE WILL NOT COVER IN THIS DELIVERY) 6
1.3 INTENDED AUDIENCE 6

CHAPTER 2: SYSTEM OVERVIEW 8

2.1 PROBLEM CONTEXT 8
2.2 SYSTEM OBJECTIVES 8
2.3 HIGH-LEVEL SYSTEM DESCRIPTION 8
2.4 USER ROLES AND STAKEHOLDERS 9

CHAPTER 3: FUNCTIONAL SPECIFICATIONS 10

3.1 FUNCTIONAL SCOPE 10
3.1.1 IN-SCOPE FEATURES 10
3.1.2 OUT-OF-SCOPE FEATURES 10
3.2 USER ROLES AND PERMISSIONS 10
3.3.1 USER AUTHENTICATION AND AUTHORIZATION 11
3.3.2 USER MANAGEMENT 11
3.3.3 CORE SYSTEM FUNCTIONS 11
3.3.4 DATA INPUT AND VALIDATION 13
3.3.5 DATA PROCESSING 13
3.3.6 DATA STORAGE AND RETRIEVAL 13
3.3.7 REPORTING AND OUTPUTS 13
3.3.8 ERROR HANDLING AND NOTIFICATIONS 13
3.4 USE CASE OVERVIEW 14
3.4.1 USE CASE DIAGRAM 14
..... 14

CHAPTER 4: NON-FUNCTIONAL REQUIREMENTS 16

4.1 PERFORMANCE REQUIREMENTS 16
4.2 SECURITY REQUIREMENTS 16
4.3 USABILITY REQUIREMENTS 17
4.4 RELIABILITY AND AVAILABILITY 17
4.5 SCALABILITY 17

CHAPTER 5: TECHNICAL SPECIFICATIONS 18

5.1 SYSTEM ARCHITECTURE 18
5.1.1 ARCHITECTURAL OVERVIEW 18
5.2 TECHNOLOGY STACK 18
5.3.1 DATA MODEL OVERVIEW 19

5.4 SECURITY DESIGN 19

CHAPTER 6: DEPLOYMENT AND ENVIRONMENT 20

6.3 NETWORK REQUIREMENTS 21

CHAPTER 7: TESTING AND VALIDATION 22

7.1 TESTING APPROACH 22

CHAPTER 8: CONCLUSION 23

List of Tables

Table 1 - Table of Definitions and Abbreviations 7
Table 2 - Table of Definitions and Abbreviations 9
Table 3 - Table of user roles and permissions 10
Table 4 -Table of accessibility preferences 11
Table 5 - Table of LinkAble Vision System Functions 12
Table 6 - Table of LinkAble Connect System Functions 12
Table 7 - Table of LinkAble Core System Functions 12
Table 8 - Table of use case diagram 15
Table 9 - Table of Performance Requirements 16
Table 10 - Table of Security Requirements 16
Table 11 - Table of Usability Requirements 17
Table 12 - Table on Reliability and Availability 17
Table 13 - Table on Scalability 17
Table 14 - Table of Component Description 18
Table 15 - Table of Technology Stack 19
Table 16 - Table of Database Schema 19
Table 17 - Tables of Hardware Requirements 20
Table 18 - Table of Software Requirements 20
Table 19 - Table of Network Requirements 21
Table 20 - Table of Test Types 22
Table 21 - Table of Acceptance Criteria 23

LinkAble Vision (navigation for visually-impaired users) and LinkAble Connect (sign-first communication for Deaf/HoH users)

This report specifies the functional and technical requirements for the LinkAble platform. It is written to be directly implementable and testable while keeping the scope realistic for an academic MVP.

Chapter 1: Introduction

1.1 Purpose of the Document

This Functional and Technical Specification (F&TS;) report defines what the LinkAble system must do (functional behaviour) and how it will be designed and implemented (technical approach). It is the baseline for development, testing, and evaluation.
LinkAble consists of two equal-focus applications:

(1) LinkAble Vision, which supports safer independent walking through voice-guided navigation and obstacle awareness; and

(2) LinkAble Connect, which supports sign-first messaging through an SLSL keyboard and an opt-in Telegram inline bot that converts between sign tokens, Sinhala text, and sign-strip images.

1.2 Scope of the System

1.2.1 In-Scope

LinkAble Vision

- iOS/Android mobile app for navigation support for blind/low-vision users.
- Turn-by-turn walking navigation with spoken prompts and rerouting.
- On-device obstacle awareness using the phone camera and audio alerts.
- Hands-free controls for key actions where feasible (start/stop/repeat).
- SOS feature that shares the user's location with predefined contacts.
- Accessibility settings (voice rate, alert frequency, high-contrast UI).

LinkAble Connect

- System-level SLSL keyboard: Android IME and iOS keyboard extension.
- Token-based output into any chat input (e.g., [[SLSL_023]]) for app compatibility.
- Local preview bar showing sign icons while typing (sign-first UX).
- On-device next-sign suggestions (MVP: n-gram/frequency model).
- Telegram inline bot (opt-in per message) for controlled conversions: tokens -> Sinhala and Sinhala -> sign-strip images, with safe unknown-word handling.
- Container app for onboarding, keyboard enablement guidance, settings, and asset updates.

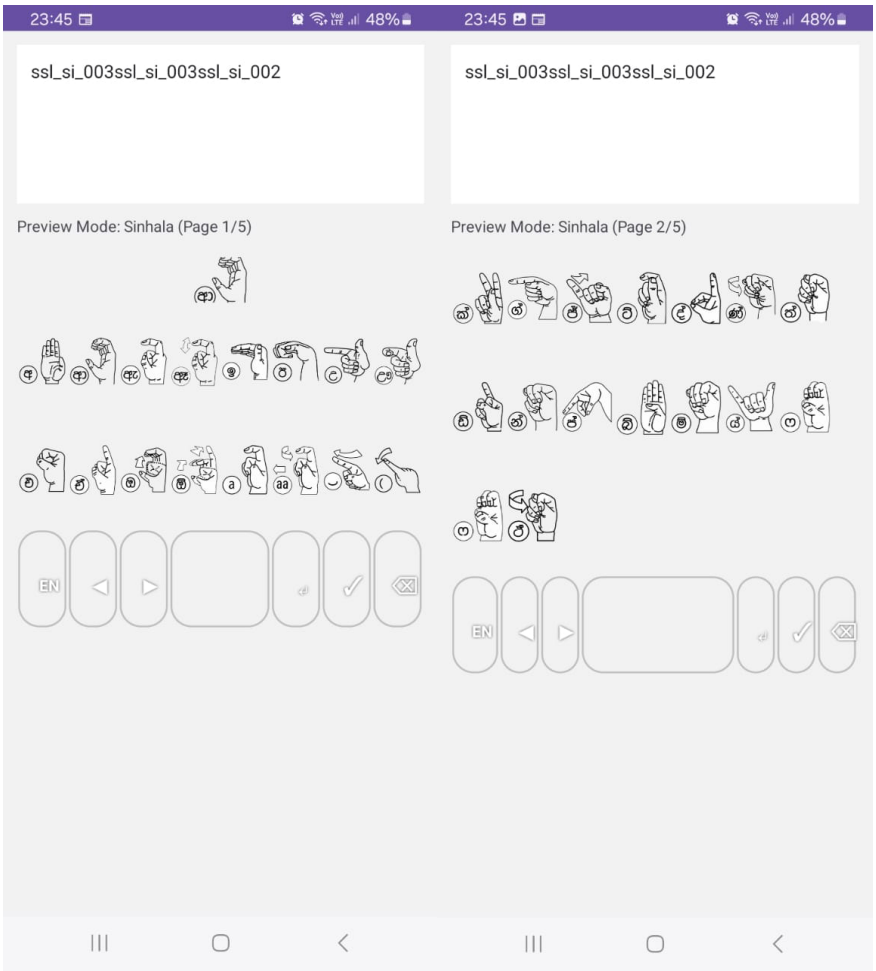

Image 01. Current status of the Linkable connect with a new addition of preview bar

Shared (LinkAble Core): configuration storage, lexicon/phrasebook management, secure APIs where needed, and privacy-preserving opt-in telemetry (optional).

1.2.2 Out-of-Scope (What LinkAble Will Not Cover in This Delivery)

- Full automatic Sinhala<->SLSL translation trained on large aligned corpora.
- Silent interception of private chats; conversions are user-triggered via inline mode.

- Deep WhatsApp/Messenger integrations beyond standard keyboard insertion.
- Wearables or external sensors as required hardware.
- 3D/animated sign avatars as mandatory output (MVP uses sign-strip images).
- Enterprise-grade scaling guarantees and commercial SLAs.

1.3 Intended Audience

- Academic supervisor(s) and evaluators (scope, feasibility, measurable outcomes).
- Developers implementing iOS/Android apps, keyboards, backend services, and bot.

- Testers/reviewers validating requirements, usability, and accessibility.
- Pilot participants and accessibility stakeholders reviewing real-world usefulness.

1.4 Definitions and Abbreviations

Term	Meaning
LinkAble	Assistive platform containing LinkAble Vision and LinkAble Connect.
LinkAble Vision	Mobile navigation and obstacle-awareness assistant for blind/low-vision users.
LinkAble Connect	Sign-first messaging system using an SLSL keyboard and Telegram inline bot.
SLSL	Sri Lankan Sign Language.
Token	Plain-text sign identifier inserted into chat inputs (e.g., [[SLSL_023]]).
Lexicon	Catalogue of signs: sign_id, gloss, category, and icon assets.
Gloss	Short label for a sign meaning (e.g., YOU, HELP).
IME	Android Input Method Editor framework for system keyboards.
iOS Keyboard Extension	Apple keyboard extension used to provide a custom system keyboard.
Inline Bot	Telegram feature where users explicitly trigger bot results inside a chat.
Sign-strip	Generated image stitching sign icons in order to represent a signed message.
GPS	Global Positioning System used for location and routing in Vision.
Rerouting	Recomputing the walking route when the user deviates.
TTS / STT	Text-to-Speech and Speech-to-Text for voice interaction.
Object/Obstacle Detection	Computer vision that identifies obstacles in the camera view for alerts.
SOS	Emergency feature that shares location with predefined contacts.
UAT	User Acceptance Testing with representative users.

Table 1 - Table of Definitions and Abbreviations

Chapter 2: System Overview

2.1 Problem Context

LinkAble Vision: Standard map apps give directions, but visually-impaired users also need real-time environmental awareness. Crowds, poles, steps, vehicles, and uneven surfaces make walking risky without timely audio guidance and a reliable emergency option.

LinkAble Connect: Mainstream messaging apps are text/voice-first and do not preserve sign grammar. Stickers/emojis are not structured language, and many translation tools interrupt real-time chat. A sign-first method is needed that works inside a mainstream platform without violating privacy.

2.2 System Objectives

Platform objectives

• Deliver two accessibility apps with consistent usability and ethical design.
• Keep the MVP deployable and measurable (clear success criteria for performance, safety, and usability).

LinkAble Vision objectives

• Provide voice-guided walking navigation with rerouting.
• Provide on-device obstacle alerts using the camera feed.
• Offer quick SOS location sharing to predefined contacts.

LinkAble Connect objectives

• Enable sign-first composition using an SLSL keyboard that outputs stable tokens.
• Increase typing speed using on-device sign suggestions.
• Provide opt-in Telegram inline conversions between tokens, Sinhala text, and sign strips, including safe handling for unknown words.

2.3 High-Level System Description

LinkAble Vision flow: user starts a session, selects a destination, receives spoken route guidance, gets obstacle alerts from the camera model, and can trigger SOS at any time.

LinkAble Connect flow: user enables the SLSL keyboard, types signs as icons (tokens inserted into Telegram), uses local preview and suggestions, and triggers the inline bot for conversions when needed.

Aspect	LinkAble Vision	LinkAble Connect
Primary users	Blind / low-vision users	Deaf/HoH and non-verbal users (plus hearing chat partners)
Core interaction	Camera + GPS + audio guidance	Sign keyboard + token typing + inline bot conversion
Main outputs	Spoken directions, obstacle alerts, SOS message	Sinhala text, sign-strip images, token messages
Key constraint handled	Safety during walking in dynamic environments	Compatibility with mainstream messaging + sign-first UX

Table 2 - Table of Definitions and Abbreviations

2.4 User Roles and Stakeholders

• Visually-impaired user (Vision): navigation, obstacle alerts, SOS.
• Deaf/HoH or non-verbal user (Connect): sign keyboard, suggestions, bot conversions.
• Hearing chat partner: receives Sinhala outputs; can request sign-strip outputs via bot.
• Caregiver/emergency contact: receives SOS alert with location.
• System administrator (project role): manages lexicon/phrasebook versions and bot configuration.

Chapter 3: Functional Specifications

3.1 Functional Scope

3.1.1 In-Scope Features

LinkAble Vision: walking navigation (voice prompts, rerouting), camera-based obstacle alerts, hands-free controls where feasible, SOS with location sharing, and accessibility settings.

LinkAble Connect: SLSL keyboard (Android IME + iOS extension) inserting sign tokens, local icon preview bar, on-device sign suggestions, Telegram inline bot conversions (tokens->Sinhala, Sinhala->sign-strip) using phrasebook + safe fallback, and a container app for onboarding/settings/asset updates.

LinkAble Core: stores lexicon and phrasebook data, supports secure configuration endpoints, and (optional) privacy-preserving opt-in telemetry.

3.1.2 Out-of-Scope Features

Full automatic Sinhala<->SLSL translation at scale, silent chat interception, deep WhatsApp/Messenger bot integrations, mandatory wearables, mandatory animated sign avatars, and enterprise SLAs are excluded from the MVP.

3.2 User Roles and Permissions

Table 3 - Table of user roles and permissions

Role	Vision permissions	Connect permissions
Visually-impaired user	Start/stop sessions; location/ camera access; receive audio; manage SOS contacts	N/A
Deaf/HoH user	N/A	Enable keyboard; type tokens; preview/suggestions; trigger bot conversions; manage settings
Hearing partner	N/A	Receive Sinhala text; optionally request sign-strip output via inline bot
Caregiver/contact	Receive SOS alert + location	Optional (future extension)
Administrator (project)	N/A	Manage lexicon/phrasebook versions and bot configuration

3.3 Functional Requirements

3.3.1 User Authentication and Authorization

- FR-AUTH-01 (Must): Core usage shall not require complex registration. Vision runs without an account; Connect keyboard and inline bot use Telegram identity and do not require a separate LinkAble login.
- FR-AUTH-02 (Should): If preference sync is implemented, users shall be authenticated securely (e.g., OTP/device-bound) and isolated from other users' data.

- FR-AUTH-03 (Must): Administrative access to lexicon/phrasebook configuration shall be restricted to authorized project administrators.

3.3.2 User Management

FR-USER-01 (Must): Users shall manage accessibility preferences.

Area	Vision settings (examples)	Connect settings (examples)
Accessibility	Voice rate/volume; alert frequency; high-contrast UI	Gloss labels
Interaction	Repeat instruction; hands-free control toggle	preview bar behaviour
Privacy	SOS contact visibility and consent notices	bot usage guidance (inline is user-triggered)

Table 4 -Table of accessibility preferences

FR-USER-02 (Must, Vision): Users shall add/edit/remove SOS contacts stored securely on-device.

FR-USER-03 (Should): Container app onboarding shall guide users to enable permissions/keyboard and explain how to use Telegram inline mode step-by-step.

3.3.3 Core System Functions

A) LinkAble Vision

ID	Requirement (Vision)	Priority	Acceptance summary
LV-FR-01	Start navigation session to a destination and speak the first instruction.	Must	Route created and first prompt delivered.
LV-FR-02	Provide turn-by-turn walking guidance with repeat instruction.	Must	Prompts update near turns; repeat works instantly.

LV-FR-03	Detect common obstacles from camera feed and issue short audio alerts.	Must	Alerts are timely and not continuously noisy.
LV-FR-04	Reroute when the user deviates beyond a threshold.	Must	Updated route delivered without restarting session.
LV-FR-05	Trigger SOS to send location to predefined contacts.	Must	Contacts receive location or user sees clear failure reason.

Table 5 - Table of LinkAble Vision System Functions

B) LinkAble Connect

ID	Requirement (Connect)	Priority	Acceptance summary
LC-FR-01	Provide SLSL keyboard (Android IME + iOS extension) with sign-icon keys.	Must	User can compose using sign keys reliably.
LC-FR-02	Insert plain-text tokens into chat input; show icons locally in preview bar.	Must	Telegram receives tokens; user sees icons; delete stays in sync.
LC-FR-03	Offer on-device next-sign suggestions (n-gram/frequency).	Should	3-5 suggestions update quickly and can be inserted.
LC-FR-04	Inline bot: tokens -> Sinhala text (primary) + optional sign-strip.	Must	Hearing partner understands output; mapping is consistent.
LC-FR-05	Inline bot: Sinhala -> sign-strip + Sinhala caption; flag unknown words safely.	Must	Partial output is allowed; unknowns are explicit (no wrong guessing).
LC-FR-06	Render sign-strip images by stitching icons and splitting long messages.	Must	Correct order and sizing; multi-part output when needed.

Table 6 - Table of LinkAble Connect System Functions

C) Shared LinkAble Core

Table 7 - Table of LinkAble Core System Functions

ID	Requirement (Core)	Priority
CORE-FR-01	Maintain lexicon: sign_id, gloss, category, icon assets, versioning.	Must
CORE-FR-02	Maintain phrasebook mappings (primary controlled conversion).	Must
CORE-FR-03	Provide dictionary/template fallback for partial conversion and unknown marking.	Should
CORE-FR-04	Cache repeated sign-strip render outputs by sequence hash.	Should

3.3.4 Data Input and Validation

• Vision shall validate camera/location permissions before starting and provide guided steps if missing.
• Vision shall warn when GPS accuracy is insufficient for safe guidance.
• Connect shall enforce a strict token format so bot parsing is deterministic (e.g., [[SLSL_###]]).
• Inline bot shall validate payload length and token correctness and return clear errors for malformed inputs.
• Container app shall validate lexicon asset integrity after updates (IDs match icon files).

3.3.5 Data Processing

• Vision processes camera frames for obstacle detection and prioritised audio alerts; routing logic updates prompts based on GPS progress.
• Connect runs suggestion generation on-device for low latency; the bot performs phrasebook lookup first, then safe fallback mapping, and renders sign strips deterministically.

3.3.6 Data Storage and Retrieval

• Local: user preferences, cached models/icons, and Vision SOS contacts (secure storage).
• Server (Core): lexicon metadata, phrasebook/dictionary mappings, bot configuration, and optional aggregated telemetry.
• Bot cache: sequence-hash -> rendered strip image to reduce repeated work.

3.3.7 Reporting and Outputs

• Vision outputs spoken navigation steps, short obstacle alerts, and SOS messages including current location.
• Connect outputs Telegram-safe tokens while typing, Sinhala text for hearing recipients, sign-strip images for deaf recipients, and explicit unknown-word notes when mapping is incomplete.

3.3.8 Error Handling and Notifications

• Clear, actionable errors for missing permissions, weak GPS, network loss, or bot downtime.
• Bot failures shall not break messaging: user can still send tokens and retry conversion later.
• Unknown Sinhala words shall be flagged explicitly; the system shall avoid confident but incorrect sign output.

3.4 Use Case Overview

3.4.1 Use Case Diagram

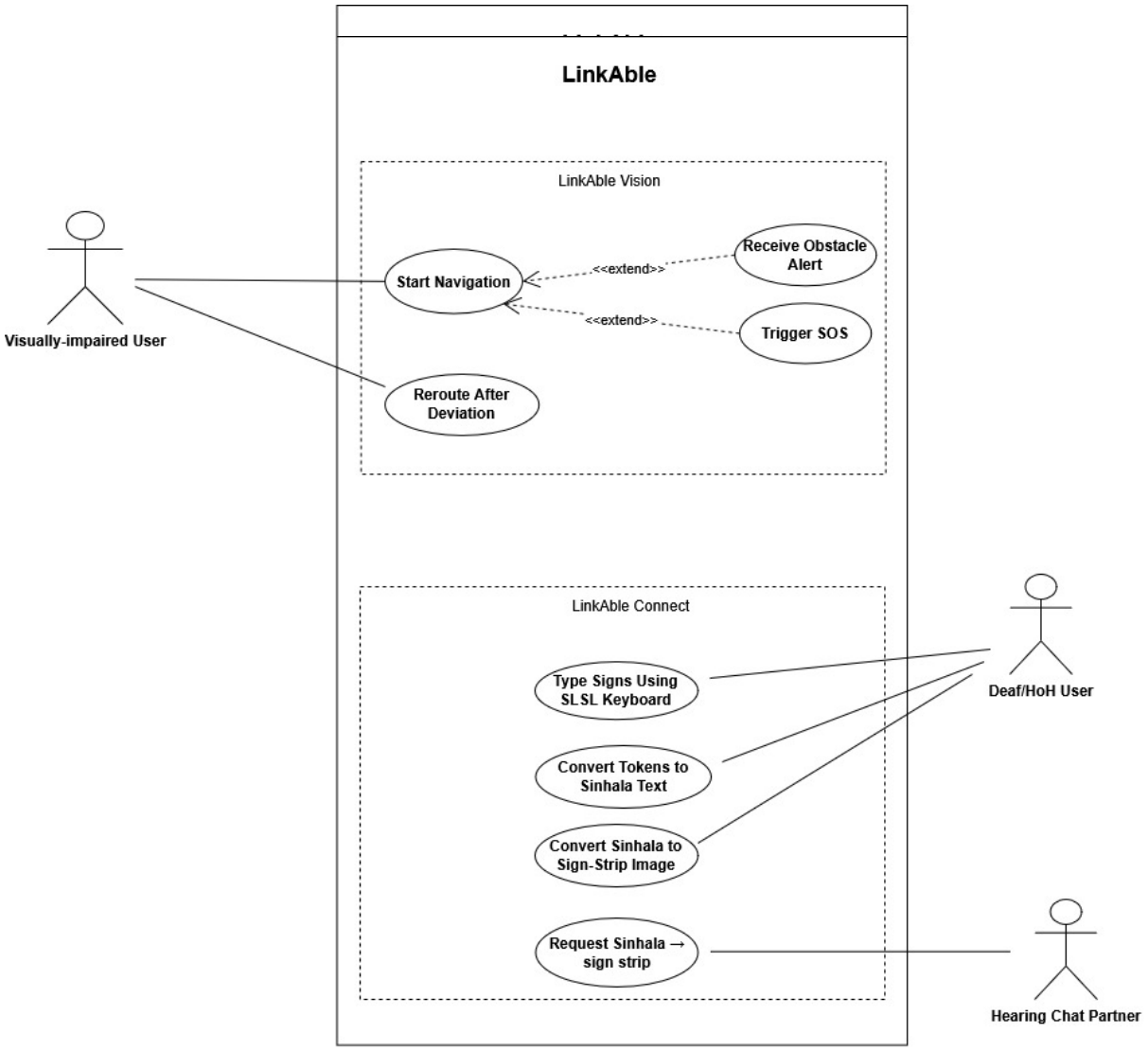

Image 02. Use Case Diagram

Note:

- "**Trigger SOS**" extends "**Start Navigation**" for diagram simplicity but is available anytime during Vision app use (per LV-FR-05).
- The diagram uses separate "**LinkAble Vision**" and "**LinkAble Connect**" boundaries to reflect the system's dual-focus architecture.

Actor	Primary use cases
Visually-impaired user	Start navigation; receive obstacle alerts; reroute; repeat instruction; trigger SOS
Deaf/HoH user	Enable keyboard; type tokens; use suggestions; convert tokens->Sinhala; convert Sinhala->sign strip
Hearing partner	Read Sinhala output; request Sinhala->sign strip via inline bot when needed
Caregiver/contact	Receive SOS alert + location

Table 8 - Table of use case diagram

3.4.2 Use Case Descriptions

- **UC-V1 Start Navigation**: Pre: location permission + GPS. Flow: select destination -> route created -> first spoken step -> guidance continues.
- **UC-V2 Obstacle Alert**: Pre: camera permission. Flow: model detects obstacle -> app issues short audio warning; user continues or stops.
- **UC-V3 Reroute**: Pre: active session. Flow: deviation detected -> new route computed -> updated spoken guidance.
- **UC-V4 SOS**: Pre: contact saved. Flow: trigger SOS -> capture GPS -> send alert + coordinates -> show confirmation/failure.

- **UC-C1 Type Signs**: Pre: keyboard enabled. Flow: select sign icons -> tokens inserted -> preview shows icons -> user edits using backspace.
- **UC-C2 Convert Tokens -> Sinhala**: Pre: token sequence. Flow: trigger inline bot -> phrasebook/fallback -> Sinhala text returned -> user inserts result.
- **UC-C3 Convert Sinhala -> Sign Strip**: Pre: Sinhala input. Flow: phrasebook first -> dictionary fallback -> render strip -> flag unknown terms -> return strip + caption.

Chapter 4: Non-Functional Requirements

These requirements define quality, safety, and trust attributes for both LinkAble Vision and LinkAble Connect.

4.1 Performance Requirements

LinkAble must respond quickly enough to support real-time navigation and smooth communication without user frustration.

Area	Requirement	Target
App UI (both)	Screen transitions and taps feel responsive.	Typical UI response < 200 ms
Vision - obstacle alerts	Obstacle alert loop remains usable while walking.	Perceived real-time; no continuous stutter
Vision - navigation	Voice prompts update with movement and at turns.	Update within 1-2 s under normal GPS
Vision - SOS	SOS completes quickly after trigger.	Send within 10 s or show failure reason
Connect - keyboard	Typing/sign selection is smooth and preview stays synced.	No visible lag; correct delete sync
Connect - bot	Inline conversion supports natural chat flow.	Typical response < 2 s for common phrases

Table 9 - Table of Performance Requirements

4.2 Security Requirements

Security focuses on protecting user data, preventing abuse (especially for bot endpoints), and ensuring safe handling of sensitive features like SOS.

Table 10 - Table of Security Requirements

Area	Requirement (applies to both apps unless stated)
Data protection	Store sensitive settings/contacts securely on-device (Keychain/ Keystore).
Transport	Use encrypted communication (HTTPS/TLS) for any server calls.
Bot safety (Connect)	Validate token format, enforce length limits, and sanitize inputs; apply rate limiting.
Least privilege	Request only required permissions and explain purpose clearly.
Data minimization	Do not store private chat content by default; keep only necessary configuration.

4.3 Usability Requirements

Usability is critical because LinkAble targets users who may rely on the system in high-pressure, real-world contexts. Usability here includes accessibility and learnability.

Area	Vision expectation	Connect expectation
Learnability	Onboarding enables first navigation session quickly.	Onboarding enables keyboard + inline bot use quickly.
Accessibility	Voice-first UI, large controls, high contrast.	Large keys, optional gloss labels, screen
Low cognitive load	Short prompts, repeat control, minimal	Fast category access, preview bar, predictable suggestions.
Error clarity	Explain GPS/camera issues and provide next steps.	Explain bot failures/ unknown words and

Table 11 - Table of Usability Requirements

4.4 Reliability and Availability

Reliability ensures LinkAble remains dependable during real usage, including poor connectivity and permission limitations.

Requirement	Expected behaviour
Crash resistance	No critical crash in standard end-to-end test runs.
Graceful offline behaviour	Vision continues obstacle alerts locally; navigation warns if routing cannot update. Connect keyboard works offline for typing/preview; bot conversions require internet with clear retry.
Recovery	If permissions or services fail, show guided recovery steps (not silent failure).
Consistency	Same token sequence yields the same mapped output when using the same lexicon/phrasebook version.

Table 12 - Table on Reliability and Availability

4.5 Scalability

Scalability is defined at an MVP/pilot level: LinkAble should support testing with real users without becoming unstable.

Table 13 - Table on Scalability

Component	Requirement (MVP/pilot scale)
Inline bot (Connect)	Handle concurrent pilot requests using caching and rate limits; timeouts should be rare.
Data growth (Core)	Lexicon and phrasebook can grow without redesign; versioned updates supported.
Deployability	Services can be separated later (bot, data, asset storage) if usage increases.

Chapter 5: Technical Specifications

5.1 System Architecture

5.1.1 Architectural Overview

LinkAble uses a hybrid design: safety- and latency-critical functions run on-device, while shared configuration and Telegram conversion logic run as lightweight services. LinkAble Vision runs camera inference and guidance locally and uses routing services for walking directions. LinkAble Connect runs keyboard input and sign prediction locally, while an opt-in Telegram inline bot performs controlled conversions and renders sign-strip images.

5.1.2 Component Description

Component	Purpose	Used by
Vision mobile app	Session UI, audio guidance, SOS trigger, routing integration.	Vision
Vision on-device AI	Obstacle detection from camera frames (quantized mobile model).	Vision
Connect keyboard (IME/ extension)	Sign-key input, token insertion, local preview bar.	Connect
Connect prediction engine	Next-sign suggestions using n-gram/ frequency model.	Connect
Telegram inline bot service	Tokens<->Sinhala controlled mapping; sign-strip rendering; unknown handling; caching.	Connect
LinkAble Core services	Lexicon/phrasebook storage, versioning, secure configuration endpoints.	Both

Table 14 - Table of Component Description

5.2 Technology Stack

Area	Technology (proposed)	Notes
Mobile apps (Vision + container)	Flutter (cross-platform)	Shared UI approach; accessibility support.
Vision routing/maps	Mapbox SDK (walking) / OSM-based routing	Turn-by-turn walking navigation with reroute.
Vision AI inference	TensorFlow Lite or ONNX Runtime Mobile	On-device inference; quantized models.
Android keyboard	Kotlin IME	Required for system-level keyboard input.
iOS keyboard	Swift keyboard extension	Required for iOS system keyboard.

Bot + APIs	Python FastAPI (REST)	Simple integration; robust for token parsing + rendering.
Sign-strip rendering	Python Pillow	Deterministic icon stitching; multi-part strips.
Database + cache	PostgreSQL + Redis (optional) + object storage	Structured mappings; fast reuse and asset delivery.

Table 15 - Table of Technology Stack

5.3 Data Design

5.3.1 Data Model Overview

• Vision data: user settings, SOS contacts, optional saved destinations, and anonymized performance metrics (opt-in).

• Connect data: lexicon (sign_id, gloss, category, icon), phrasebook mappings (primary), dictionary/template fallbacks, and rendered-strip cache.

• Core principle: minimize sensitive data; do not store private chat messages by default.

5.3.2 Database Schema

Table 16 - Table of Database Schema

Entity/table	Key fields (summary)	Used by
devices	device_id, platform, app_version	Both
vision_settings	device_id, voice_rate, alert_sensitivity, ui_mode	Vision
emergency_contacts	device_id, channel, address/number, priority	Vision
lexicon_signs	sign_id, gloss, category, icon_uri, version, active	Connect
phrasebook_entries	direction, source, target, sequence_hash, updated_at	Connect
dictionary_entries	language, word_norm, sign_id	Connect
render_cache	sequence_hash, image_uri, hit_count	Connect
consent_logs	device_id, consent_type, status, timestamp	Both

5.4 Security Design

• Encrypt in transit (HTTPS/TLS) for any external calls (routing, bot, core APIs).
• Store sensitive local data securely (Keychain/Keystore), especially SOS contacts and settings.
• Bot hardening: strict token parsing, payload length limits, input sanitization, rate limiting, and safe error responses.
• Privacy controls: inline bot is user-triggered per message; opt-in analytics are aggregated and anonymized.
• Versioning: lexicon and phrasebook versions are tracked to keep conversion results consistent across updates.

Chapter 6: Deployment and Environment

6.1 Hardware Requirements

Device	Minimum	Recommended	Notes
Android phone	64-bit, 4GB RAM, GPS, rear camera, speaker	6-8GB RAM, stronger CPU/GPU	Vision needs stable camera+compute; Connect keyboard must be smooth.
iPhone	Modern iOS version supporting keyboard extensions	Newer iPhone models	Required for iOS keyboard testing and smooth Vision processing.
Audio (optional)	Earphones	Bone-conduction headset	Helps Vision guidance while preserving ambient awareness.

Server (bot/core)	Minimum (pilot)	Recommended
Compute	2 vCPU, 4GB RAM	4 vCPU, 8GB RAM
Storage	20GB SSD	40GB SSD
Caching	Optional	Recommended (render cache)

Table 17 - Tables of Hardware Requirements

6.2 Software Requirements

Table 18 - Table of Software Requirements

Area	Requirement
End-user apps	Android/iOS with accessibility services enabled; Telegram installed for Connect.
Vision permissions	Location + camera (and microphone only if voice commands are implemented).
Connect enablement	Keyboard extension/IME enabled; container app installed for onboarding/settings.
Server	Linux host for bot/core services, HTTPS termination (e.g., Nginx), database (PostgreSQL), and optional cache (Redis).

6.3 Network Requirements

Table 19 - Table of Network Requirements

Component	Connectivity need	Behaviour under weak network
Vision routing	Mobile data/Wi-Fi recommended for route fetch and reroute.	Warn user; continue obstacle alerts locally.
Connect keyboard	Not required for typing tokens/ preview.	Works offline; bot conversion requires internet.
Inline bot	Internet required to request conversion results.	Show retry guidance; do not block token sending.

Chapter 7: Testing and Validation

7.1 Testing Approach

Testing verifies that LinkAble meets its functional requirements and is safe, usable, and trustworthy for its target users. Vision testing focuses on navigation correctness, timely obstacle alerts, and reliable SOS behaviour. Connect testing focuses on keyboard correctness (tokens + preview), controlled conversions, and safe handling of unknown words.

7.2 Test Types

Table 20 - Table of Test Types

Test type	Vision examples	Connect examples
Unit tests	Route state updates; SOS message formatting; permission checks	Token parsing; phrasebook lookup; image stitching order
Integration tests	GPS + routing API; camera pipeline + alert output	Keyboard -> Telegram input; bot API -> Telegram inline results
System (E2E) tests	Start session -> navigate -> reroute -> SOS	Type signs -> preview sync -> tokens->Sinhala -> Sinhala->strip
Usability/ accessibility	Voice prompt clarity; large controls; TalkBack/VoiceOver labels	Key size; gloss labels; screen reader labels; low cognitive load
Performance	Alert latency under walking use; reroute response time	Keyboard lag; bot response time; cache hit effectiveness
Security/privacy validation	No unnecessary permission use; secure SOS storage	Inline-only conversion; rate limiting; no chat storage by default

7.3 Acceptance Criteria

Area	Acceptance criteria (MVP)
Vision functional	Navigation starts and provides spoken prompts; rerouting works; repeat instruction works; SOS sends location or shows clear failure.
Vision safety/usability	Obstacle alerts are understandable and not continuously noisy; app warns when GPS accuracy is low.
Connect functional	Keyboard inserts valid tokens; preview bar stays in sync with edits; category browsing works; suggestions insert correctly.

Connect conversion	Inline bot returns Sinhala for common token sequences; Sinhala->strip works; unknown words are explicitly flagged (no misleading translation).
Cross-cutting quality	No critical crashes in standard E2E runs; clear errors for permissions/network; sensitive data stored securely; opt-in analytics only (if used).

Table 21 - Table of Acceptance Criteria

Chapter 8: Conclusion

To sum up, the LinkAble Functional and Technical Specification Report sets a feasible and verifiable MVP for an assistive technology platform featuring two equal mainstays: LinkAble Vision (voice-guided walking navigation, on-device obstacle alerts, and SOS location sharing for the blind/low-vision users) and LinkAble Connect (a sign-first SLSL keyboard with token-based typing, on-device next-sign suggestions, and an opt-in Telegram inline bot that performs controlled conversions between tokens, Sinhala text, and sign-strip images with safe unknown-word handling). The report establishes a clear distinction between in-scope and out-of-scope features to ensure deliverability. It also sets measurable requirements in terms of functionality and non-functionality (performance, security, usability, reliability) and puts forward a hybrid architecture where latency- and safety-critical tasks are performed on-device while shared lexicon/phrasebook services and bot logic are executed server-side with privacy-preserving design choices. In a nutshell, this document offers an excellent basis for the implementation, testing, and evaluation of the project while making accessibility, safety, and ethical data practices the top priority.